Rasterizarea suprafetelor poligonale 2D

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Suprafete poligonale 2D

- Definite prin lista coordonatelor varfurilor.

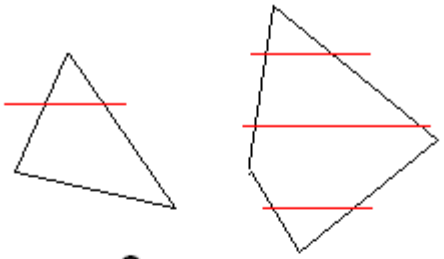
Rasterizarea unei suprafețe poligonale 2D: descompunerea suprafetei în fragmente care se afiseaza in pixeli.

- **Necesita calculul adreselor pixelilor in care se afiseaza fragmentele**
- In cazul unei suprafețe poligonale 3D transformată în spațiul de afişare, algoritmul de rasterizare include:
  - Calculul coordonatei z pentru fiecare fragment (explicat in cursul despre Z-buffer)
  - Calculul culorii fragmentului în fragment shader
  - Testul de adancime

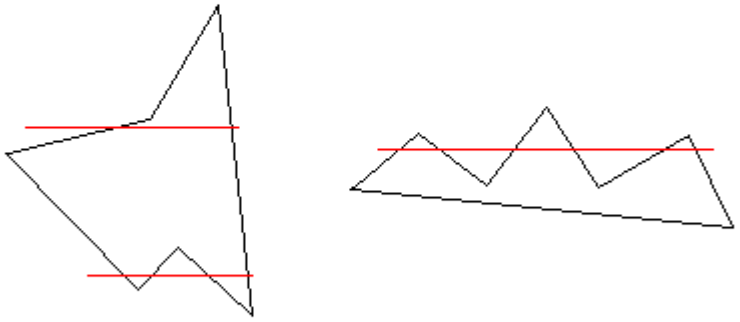
  Aceste operatii le consideram incluse in functia putpixel (int x, int y)
- Pentru o suprafață poligonală definită într-un sistem grafic 2D vom folosi pentru afişarea unui fragment funcția setpixel (in x, int y, RGB culoare).

Rasterizarea suprafetelor poligonale (1)

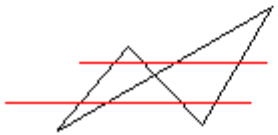
O suprafață triunghiulară este un caz particular de suprafață poligonala.



Convexe



Concave



Autointersectant

Forma generală a algoritmului:

pentru toate liniile raster care intersecteaza suprafata poligonului

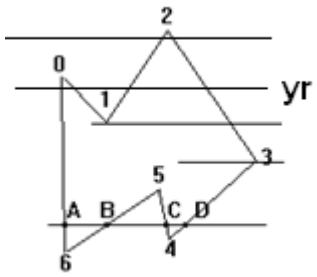
- calculeaza punctele de intersectie dintre linia raster si laturile poligonului
- coloreaza pixelii de pe segmentele interioare poligonului de pe fiecare linie raster

Probleme

1) O linie raster nu intersecteaza toate laturile poligonului: nu trebuie calculate intersectiile cu aceste laturi.

2) Intersectia dintre o linie raster si un poligon concav produce mai multe segmente de pixeli interiori.

Rasterizarea suprafetelor poligonale (2)



1) O linie raster nu intersecteaza toate laturile poligonului.

➢ se determina pentru fiecare linie raster setul laturilor intersectate:

setul laturilor active (ymin_latura <= yr < ymax_latura)

2) Intersectia dintre o linie raster si un poligon concav produce mai multe segmente de pixeli interiori. Pentru determinarea segmentelor de pixeli interiori de pe o linie raster, yr:

➢ Se ordonează punctele de intersecţie crescător după abscisă: x1< x2< x3 <x4 .

Exemplu: xA< xB< xC< xD.

➢ Segmentele de pixeli interiori suprafetei poligonului sunt:

(x1, yr) – (x2,yr) şi (x3-yr) – (x4,yr). Exemplu: (A-B, C-D).

❖ Punctele de pe segmentele (x1, yr) – (x2,yr), (x3-yr) – (x4,yr), ..,respecta **definitia punctului interior unui poligon**: "***un punct este interior unui poligon dacă orice semidreaptă dusă din punctul respectiv intersectează poligonul într-un număr impar de puncte*** "

Rasterizarea suprafetelor poligonale (3)

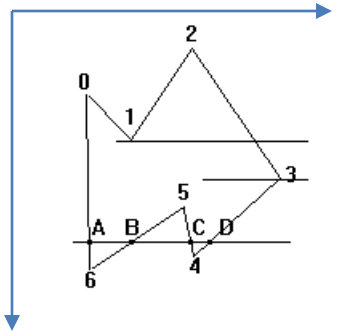
❖ **Cazuri particulare: liniile raster care traverseaza varfurile poligonului:**

- daca fiecare varf apartine la 2 laturi, atunci, o semidreapta orizontala dusa dintr-un punct interior poligonului

  -va produce 3 puncte de intersectie daca traverseaza varful 1

  -va produce 2 puncte de intersectie daca traverseaza varful 3

- Varfurile 0, 1, 2, 4, 5, 6 sunt varfuri de minim / maxim local



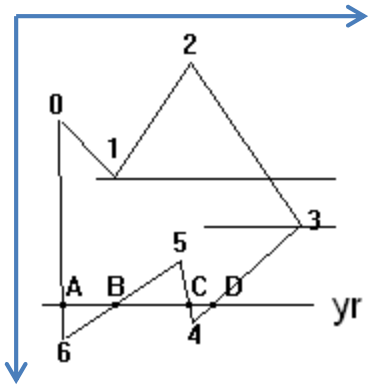
Conventie:

- la traversarea unui varf de minim / maxim local trebuie sa rezulte 0 sau 2 puncte de intersectie
- la traversarea oricarui alt varf trebuie sa rezulte un singur punct de intersectie

Implementarea conventiei: se considera fiecare latura un interval inchis la un capat si deschis la celalalt, de exemplu: [ymin_latura – ymax_latura). Atunci:

-un vârf de maxim local nu aparţine nici uneia dintre laturile care-l formează iar un vârf de minim local aparţine ambelor laturi.

Rasterizarea suprafetelor poligonale (4)



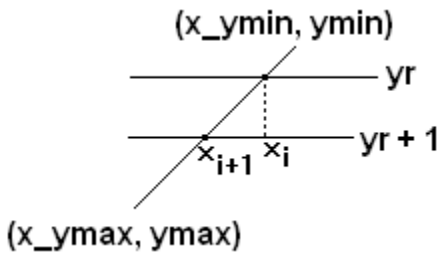
Algoritmul de rasterizare

1) Se crează lista laturilor poligonului, ordonată crescător după ordonatele minime ale laturilor: 1-2, 2-3, 0-1, 6-0, 3-4, 4-5, 5-6

2) for(yr = ymin_poligon; yr< ymax_poligon; yr++)

- Determina setul laturilor active pe linia y=yr; exemplu: 6-0, 3-4, 4-5, 5-6.
- Calculează intersecţiile dreptei y=yr cu laturile active, **in ordinea din lista laturilor**; pentru exemplul din figura punctele de intersectie vor fi: A, D, C, B.
- Ordonează punctele de intersecţie crescător după abscisă: A, B, C, D.
- Colorează pixelii de pe segmentele de dreaptă interioare suprafetei poligonului (A-B, C-D).

Rasterizarea suprafetelor poligonale (5)



Calculul intersectiilor liniilor raster cu laturile poligonului

$m = (ymax - ymin)/(x_ymax - x_ymin) = (yr+1 - yr)/(x_{i+1} - x_i)$

$x_{i+1} = x_i + 1/m$ sau $x_{i+1} = x_i + dx$

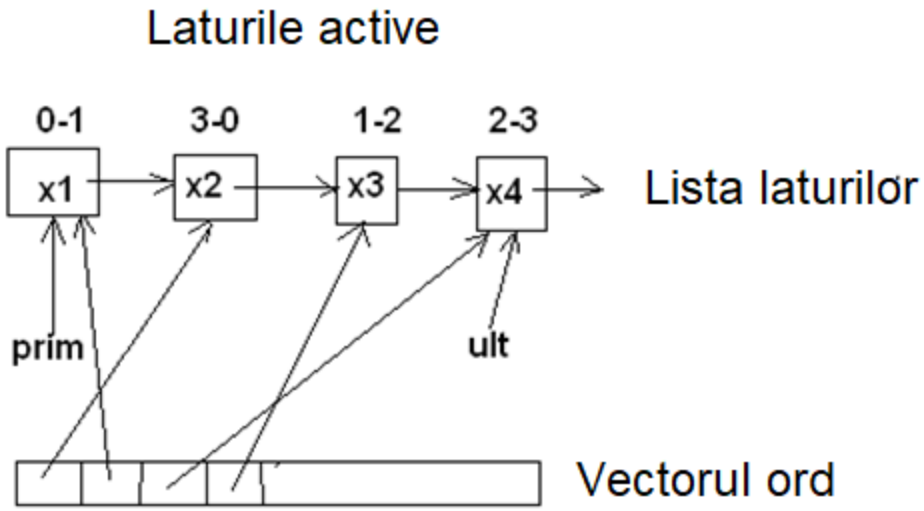
x1 = x_ymin – dx (inainte ca latura sa devina activa)

Implementare-1

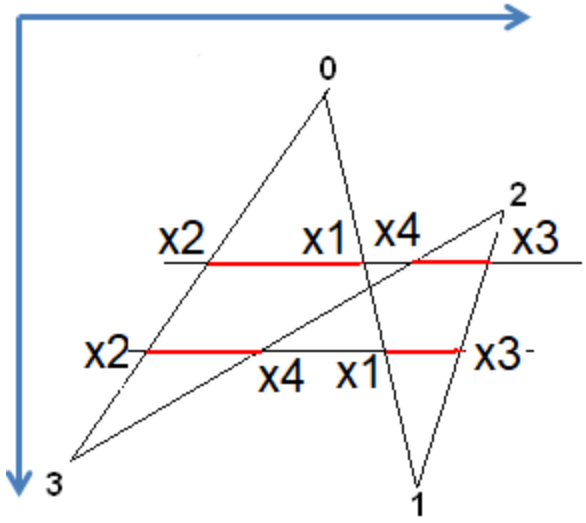
Reprezentarea unei laturi in lista de laturi:

- ymax_latură şi ymin_latură, folosite în testul de latură activă;
- xi - abscisa ultimului punct de intersecţie al laturii;
  initial, xi = abscisa varfului de ymin_latura - dx
- dx constanta folosită în calculul incremental al absciselor punctelor de intersecţie.

Implementare-1 (2)




```
typedef struct {int x,y;} virf;
struct latura{  int ymax,ymin;
                float xi,dx;
                struct latura * urm;
              };
typedef struct latura latura;
latura * prim,*ult;
latura **ord;
```

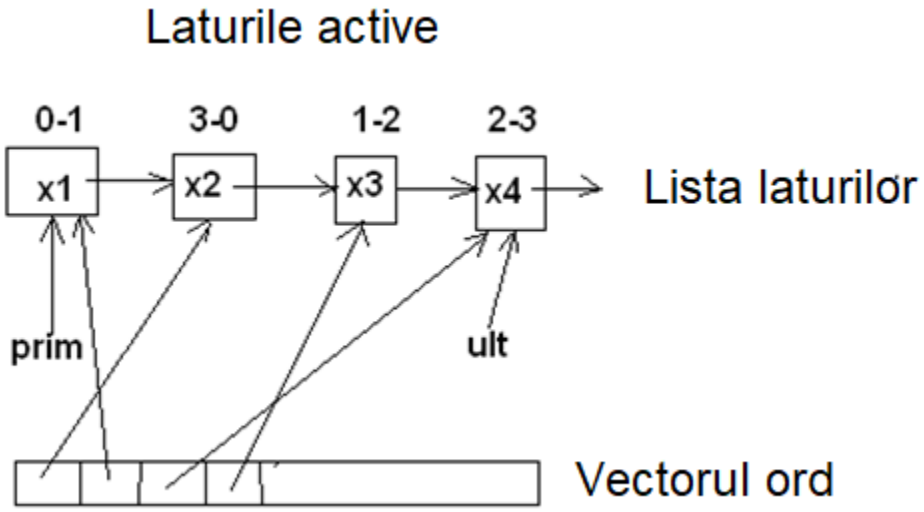


- prim si ult delimiteaza setul laturilor active in lista laturilor
- vectorul ord contine pointeri catre punctele de intersectie in ordinea crescatoare a absciselor

➢ Vectorul ord trebuie ordonat pentru fiecare linie raster,

chiar daca nu s-a schimbat setul laturilor active.

Implementare-1 (3)



Determina lista laturilor active
pentru linia raster curenta

Initial: ult=prim;

int set_activ(int yr)

{

* Adaugă laturile care devin active
(avans **ult** in lista laturilor cat timp yr ==ymin_latura)
* Elimină din lista laturilor, intre prim si ult, laturile care au devenit inactive:
(yr == ymax_latura)

return 1 daca s-a modificat setul laturilor active,
0 altfel;

}

Implementare-1 (4)

int colorare_poligon(int nrv, varf poly[])

{ // **nrv** este numarul de vârfuri ale poligonului; **poly** este vectorul vârfurilor

latura **ord; latura * k; int I, k, modif, x;

* **Creaza lista de laturi ordonata crescator dupa ymin_laturi**;

ult = prim;

// Genereaza interiorul poligonului prin intersectii cu linii raster, de la ymin_poligon la

// ymax_poligon

for(yr=ymin_poligon; yr < ymax_poligon; yr++)

{ modif=set_activ(yr); // calculeaza setul laturilor active

*** Calculeaza punctele de intersecţie cu laturile active (memorate in celulele listei de laturi intre prim si ult): xi = xi + dx**

if(modif) // s-a modificat setul laturilor active

*** Memoreaza in vectorul ord pointeri catre laturile active**

Implementare-1 (5)

// Ordoneza vectorul ord a.i. sa puncteze catre punctele de intersecţie cu laturile active in ordinea crescatoare a absciselor

```
do  {o=0;
      for(i=0; i<nrint-1; i++) //  nrint este numarul de puncte de intersectie
           if(ord[i]->xi > ord[i+1]->xi) {k=ord[i]; ord[i]=ord[i+1]; ord[i+1]=k;o=1;}
 }while(o);
```

// Coloreaza pixelii interiori poligonului de pe linia curenta

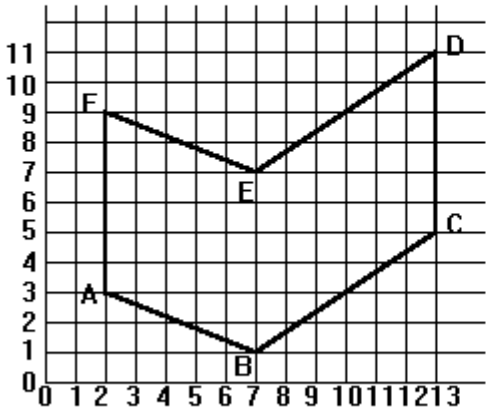
```
for(i=0; i<nrint-1; i+=2)
  for(x = (int)(ord[i]->xi+0.5); x<=(int)(ord[i+1]->xi+0.5); x++)
  { putpixel(x,yr)  sau setpixel(x, yr, culoare(x,yr)); // afiseaza fragmentul
}//for yr
return 1;
```

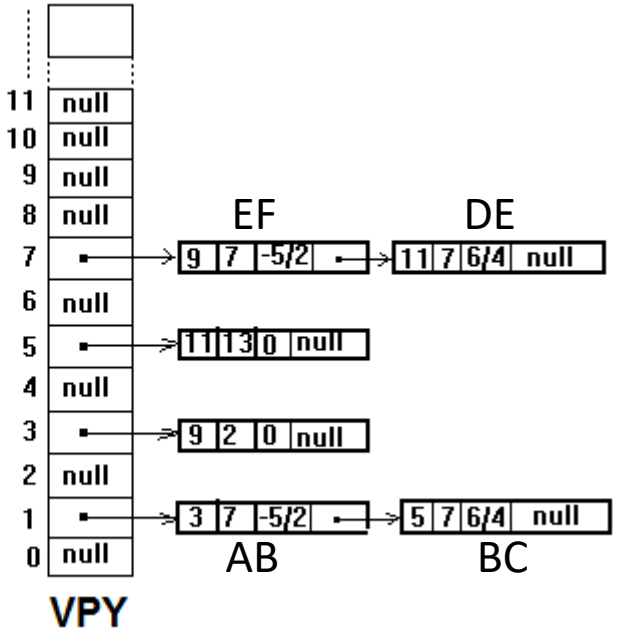
setpixel (int x, int y, RGB culoare) – scrie in frame buffer culoarea pixelului (x,y)
-pentru un poligon definit intr-un sistem grafic 2D

RGB **culoare**(int x, int y) – intoarce culoarea calculata pentru fragmentul afisat in pixelul (x,y)

Implementare-2 (1)

Poligonul ABCDEFA in spatiul ecran





- **VPY**: vector de pointeri cu un număr de intrări egal cu numărul de linii imagine.
- **VPY[i]**: pointer către lista laturilor care au ordonata minimă (ymin) egală cu **i**
- Pentru fiecare latură se memorează: ordonata maximă , abscisa vârfului de ordonată minimă şi incrementul (1/m) folosit în calculul punctelor de intersecţie.
- Lista laturilor cu acelaşi ymin este ordonată crescător după abscisa vârfului cu ordonata ymax.

Implementare-2 (2)

Lista laturilor active (LLA) este reprezentată separat, printr-o listă înlănţuită, în care laturile sunt memorate în ordinea crescătoare a absciselor punctelor de intersecţie cu linia curenta.

Actualizare LLA pentru linia raster curenta, yr:

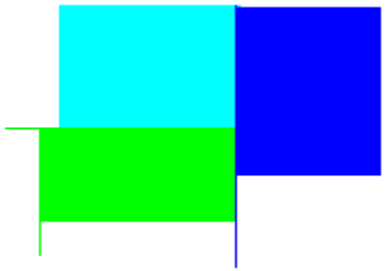
- Se adauga la LLA lista de laturi punctata de VPY[yr]: laturi care devin active
- Se elimina din LLA acele laturi pentru care yr == ymax_latura
- Se sorteaza LLA crescator dupa x-intersectie (calculat incremental pornind de la x-ymin)

Comparatie cu implementare-1

- Lista laturilor se construieste mai rapid dar consuma mai multa memorie (în majoritatea cazurilor numai un număr mic dintre intrările vectorului VPY sunt folosite)
- Actualizarea listei de laturi active consuma mai mult timp.

Generarea pixelilor strict interiori

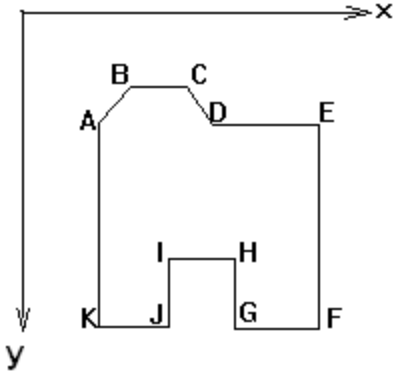
Trebuie evitata suprascrierea pixelilor de pe laturile comune poligoanelor .



Conventie:

-Pixelii de pe laturile de xmin si ymin apartin poligonului (se afiseaza)

-Pixelii de pe laturile de xmax si ymax nu se afiseaza



Datorita conventiei folosite pentru traversarea varfurilor poligonului (Implementare 1):

- Liniile orizontale care unesc 2 varfuri de minim local vor fi afisate
- Liniile orizontale care unesc 2 varfuri de maxim local nu vor fi afisate

➢In acest fel se respecta conventia pentru laturile orizontale.

➢Se afiseaza: BC, DE, IH

Generarea pixelilor strict interiori

Pentru laturile neorizontale:

Pentru punctele de xmin si xmax ale segmentelor interioare poligonului:

- daca fiecare segment interior poligonului este un interval inchis in punctul de abscisa minima si deschis in punctul de abscisa maxima, atunci se respecta conventia.

```
for(x = (int)(ord[i]->xi+0.99999); x<=(int)(ord[i+1]->xi-0.000001); x++)
    setpixel(x, yr, culoare(x,yr));
```

❖ Generarea numai a pixelilor strict interiori nu dă rezultate bune atunci când unghiul dintre două laturi este foarte mic. Este posibil ca între cele două laturi să fie afişat un singur pixel sau chiar niciunul.

Generarea interiorului unui poligon folosind un şablon

Modificarea implementarii anterioare pentru generarea interiorului folosind un sablon

```
int sablon[8][8]={255,0,0,0,0,0,0,255, 0,255,0,0,0,0,255,0, 0,0,255,0,0,255,0,0,
                  0,0,0,255,255,0,0,0,0,0,0,255,255,0,0,0, 0,0,255,0,0,255,0,0,
                  0,255,0,0,0,0,255,0,255,0,0,0,0,0,0,255
                }; //matrice de intensităţi

int hs=8, ls=8, lin, x;//hs – inaltimea sablonului; ls – latimea sablonului
// sablonul se multiplica incepand din (x=0, y=0): originea imaginii

void colorare_poligon(int nrv, virf poly[] )
{……..
 for(yr=ymin_poligon; yr < ymax_poligon; yr++)
 { ……
   lin=yr%hs; // linia din matricea sablon folosita la colorarea poligonului pe linia raster yr
   for(i=0;i<nrint-1;i+=2) // nrint: numarul de intersectii dintre linia raster si suprafata
                           // poligonului
     for(x=(int)(ord[i]->xi+0.99999; x<=(int)(ord[i+1]->xi+0.000001); x++)
           setpixel(x,yr,sablon[lin][x%ls]);//afisare pixel in culoarea din sablon[lin][x%ls]
  }
```